SMART BEINGS · WOOHOO PLATFORM

MCP, Data & **Automation Integration**

Connecting the hotel/room/device data, and connecting the room automation systems — what the agent needs, what it must never see, and how to build an adapter without a hotel.

MCP | Catalogue & schema | INNCOM / Interel | Simulator

FOR
Data & integration engineers

COVERS
MCP servers · adapters · edge

TEST WITH
Full simulator, no hardware

READ FIRST

What exists, and what doesn't

This guide covers two jobs that meet in the middle: **getting hotel data into the system**, and **getting commands out to the room**. Before either, three honest statements about the current build.

Thing	Status	Detail
MCP semantics registry, per-room filtering, scope injection, sanitisation, error shaping, audit	BUILT	All of it works and is tested. This is the valuable part.
MCP protocol JSON-RPC, Streamable HTTP, an actual MCP client	NOT BUILT	The gateway calls in-process functions. There is no MCP wire protocol anywhere in the codebase. **Your existing MCP servers are not connected.** §3 is the work.
Automation adapter contract	BUILT	Interface complete, exercised by a full simulator with failure injection.
Real vendor adapters INNCOM, Interel, BACnet, KNX	NOT BUILT	Needs vendor protocol documentation and lab hardware. Part IV is how to write one.
Database	SCHEMA ONLY	Full Postgres schema with row-level security exists; repositories are in-memory. Swapping is a constructor change — §7.
Edge gateway in the actuation path	NOT WIRED	The appliance exists and is tested, but the cloud currently talks to the adapter directly. §18.

> THE ONE IDEA THAT GOVERNS EVERYTHING IN THIS GUIDE
>
> **The agent reasons about capabilities, never about vendors.** It knows there is "a thermostat in the bedroom zone with a permitted range". It has never heard of INNCOM, Interel, BACnet objects or KNX group addresses.
>
> Every vendor-specific fact lives in exactly two places: an *adapter*, and a per-property *capability manifest*. If vendor vocabulary ever appears above those, the abstraction is broken and adding the next hotel becomes an engineering project again.

Contents

Part I — MCP

1 What MCP means here
2 The gateway's six jobs
3 Connecting your MCP servers
4 What must never be an MCP tool

Part II — Data

5 The domain model
6 What the agent needs per turn
7 Schema and repositories
8 Tenant isolation
9 Caching and freshness

Part III — Capabilities

10 The capability model
11 Writing a manifest
12 Property configuration

Part IV — Automation adapters

13 The contract
14 Writing an adapter
15 Capability degradation
16 Failure, retries, circuit breakers
17 Verify-read

Part V — On premises

18 Why the edge gateway exists

19 What runs on it

Part VI — Practical

20 The simulator

21 Conformance testing

22 Onboarding a property

23 Checklist

PART I

MCP

1. What MCP means here

MCP is a protocol for exposing capabilities **to a model**. In Woohoo it is an *internal* plane: the agent proposes tool calls, and a gateway executes them. It is not a public API, and no external system calls it.

```
AGENT ──► MCP GATEWAY ──┬──► property catalogue      (rooms, devices, manifests)
proposes    the control ├──► device state    ──► automation adapter ──► the room
only        point       ├──► service requests
                        ├──► F&B orders
                        ├──► PMS bridge
                        ├──► knowledge / FAQ
                        ├──► guest preferences
                        └──► notification / escalation

The agent can NEVER address any of these directly. That single rule is what makes
tenant scoping mechanical instead of a code-review question.
```

THE GAP YOU NEED TO KNOW ABOUT BEFORE PLANNING

The gateway currently dispatches through an in-process map of handler functions:

```
const result = await gateway.execute(scoped, ctx);   // a function call
```

There is **no MCP client, no JSON-RPC, no Streamable HTTP** in the codebase. So the MCP servers you already have — hotel, room, device, services — are **not connected to this system**. The gateway was designed to front them, but today it fronts local functions. §3 is that work.

2. The gateway's six jobs

Understanding these explains why the agent is not simply pointed at your MCP servers directly — which is what MCP is for, and would be right for a single-tenant developer tool.

Job	What happens without it	What the gateway does
Scope injection	The room to act on comes from *model output*. A model error or an injected instruction can address another guest's room. Nothing structural prevents it.	Strips any room/property/tenant field the model produced and injects the real one, derived from the authenticated device.
Per-room filtering	Every hotel sees every tool. A room without curtains gets a curtain tool the model will happily call.	Filters the advertised tool set by that room's capability manifest before the agent sees it.
Result sanitisation	A menu description edited by a hotel employee becomes a prompt-injection vector.	Fences untrusted content and scans it before it enters the model's context.
Error normalisation	`INNCOM: ERR_0x8004` reaches the model, which either hallucinates an explanation or apologises uselessly.	Maps everything into one small vocabulary the model is taught to handle.
Caching	The catalogue is read on every turn and changes weekly — the hottest and most wasteful query in the system.	Per-tool TTLs with explicit invalidation.
Uniform audit	Each server logs differently, or not at all.	One place records every call, its arguments digest, its verdict, latency and outcome.

2.1 Scope injection, concretely

```
{ // what the model emitted — note the room_id it invented
  "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "targetC": 23.0, "room_id": "999" }
}

{ // what actually executed
  "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "targetC": 23.0 },        // room_id STRIPPED
  "scope": {                                            // injected server-side
    "tenantId":   "t_demo_apac",
    "propertyId": "p_blr_whitefield",
    "roomId":     "r_412",
    "deviceId":   "d_412a"
  },
  "idempotencyKey": "sha256(turnId | tool | canonical args)"
}
```

> **STRIPPED, NOT VALIDATED — AND THE DIFFERENCE MATTERS**
>
> Validating a model-supplied `room_id` against the real one would mean the field exists in the surface. Which means it can be populated. Which means somebody will eventually "temporarily" trust it for a staff feature. Removing the field removes the class of bug.
>
> **Design your MCP tools the same way: no tool takes a room, property or tenant identifier.** Scope arrives out of band.

3. Connecting your MCP servers

This is the integration work. Everything above the transport already exists — you are implementing one interface.

```
// services/woohoo-core/src/modules/mcp-gateway.ts
export type ToolHandler = (
  call: ScopedToolCall,   // tool, args, scope, turnId, idempotencyKey
  ctx:  TurnContext,
) => Promise<unknown>;
```

Today a handler is a local function. To use a real MCP server, it becomes a client call:

```
/**
 * Bridges one Woohoo tool to a tool on a real MCP server.
 *
 * Everything above this — filtering, policy, audit — is unchanged. The gateway does not
 * care whether a handler is local or remote.
 */
function mcpHandler(client: McpClient, remoteToolName: string): ToolHandler {
  return async (call, ctx) => {
    const res = await client.callTool({
      name: remoteToolName,
      arguments: {
        ...call.args,
        // Scope travels OUT OF BAND, not as tool arguments (see §2.1).
      },
      // Per-call, audience-bound, short-lived. A compromised server must not be able
      // to replay this token against a different server.
      _meta: { woohooScope: call.scope, turnId: call.turnId },
    }, { timeoutMs: 2500 });

    if (res.isError) {
      // Normalise. Raw upstream errors must never reach the model.
      return toolError(
        ToolErrorCode.TemporarilyUnreachable,
        'That system is not responding right now.',
        'Tell the guest you could not complete it and offer the front desk.',
        true,
      );
    }
    return res.content;
  };
}
```

3.1 What the MCP server must guarantee

Requirement	Why
Honour the scope, don't trust arguments	The server must resolve which room from the out-of-band scope, and reject or ignore any room identifier in the arguments. This is the single most important requirement.
No cross-tenant reachability	Given scope for property A, it must be impossible to read or write property B — even with a crafted argument.
Bounded latency	Return within ~2.5 s or fail. The turn has an 8-second wall clock and a guest is standing there.
Idempotency	Honour `idempotencyKey` on mutations. Retries happen; double actuation must not.
Structured errors	Distinguish "not available in this room" from "upstream is down" — they produce different things for the guest to hear.
No side effects on read	Read tools are pre-fetched speculatively on most turns, before anyone knows whether they are needed.

3.2 Transport

Option	Use when	Notes
In-process	Today	Zero latency. Same interface, so promotion later is a config change.
Streamable HTTP	Separately deployed servers	Recommended. Single stable endpoint, streaming results, works in-VPC over mTLS.
stdio	Local dev only	Avoid in production — couples process lifecycles and makes scaling and observability awkward.
Third-party servers	**Not yet**	A customer-supplied MCP server is a security decision, not an integration one. It needs sandboxing, separate egress policy, maximally untrusted results, and no access to anything that costs money or moves hardware.

4. What must never be an MCP tool

AUTOMATION ADAPTERS ARE NOT MCP SERVERS, AND THIS IS A TEMPTING MISTAKE

The symmetry is appealing: if the catalogue is an MCP server, why not `inncom-adapter`?

Because MCP exposes capabilities **to a model**. If the INNCOM adapter were an MCP server, its vendor-shaped tools would appear in the model's world, and within two sprints the agent would be reasoning about object IDs and register writes. The abstraction that makes a new hotel a config change would be dead.

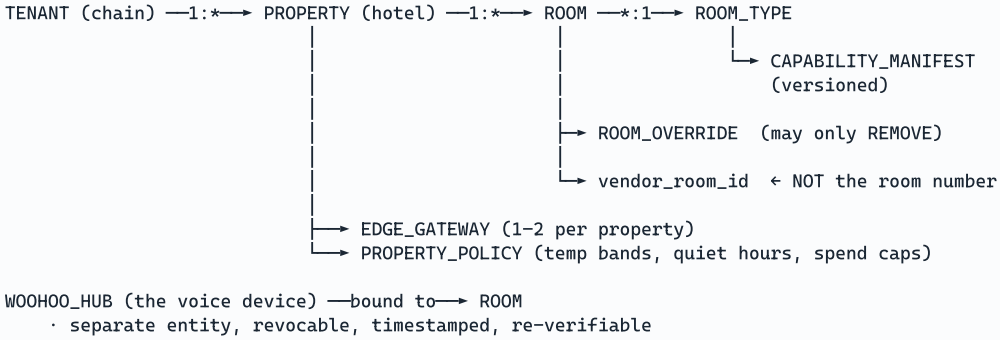
Adapters sit *behind* the device-state server, which presents one vendor-neutral surface. Adding Schneider or Lutron means writing an adapter and a mapping — the model's tool list, the prompts and the tests all stay untouched.

Never expose as a tool	Instead
Anything vendor-shaped — object IDs, registers, group addresses	A capability behind an adapter
Anything taking a room, property or tenant parameter	Ambient scope
Anything operating on more than one room	Staff plane, human-authenticated, with a cap on rooms per operation
Door locks, safes, microphone activation	Nothing. No tool exists at any tier, deliberately.
Raw SQL or arbitrary query execution	Purpose-shaped read tools
Anything handling a card number	Folio posting via the PMS only

PART II

Data

5. The domain model



TWO MODELLING DECISIONS WORTH UNDERSTANDING BEFORE YOU TOUCH THE SCHEMA

The device is not the room. Modelling the hub as a separate entity with an explicit, timestamped binding is what makes mis-binding detectable. Collapse them — treat device ID as room ID — and a device moved during a refurbishment silently controls the wrong room while the audit log records it as correct. That bug surfaces nowhere unless you build for it.

The manifest attaches to room type, not room. A 300-room property has perhaps eight types. Eight manifests plus a handful of overrides is a day's work; three hundred is a project. This single choice is most of the difference between five-day and five-week onboarding.

6. What the agent needs per turn

Five reads, all fired **in parallel while the guest is still speaking**. Together they remove an entire tool round-trip — roughly 400–600 ms — from most turns, so their latency is on the critical path.

Read	Budget	Cache	Returns
Device resolution	<5 ms	60 s	`deviceId` → tenant, property, room, manifest, vendor room ref. **Must not cost a DB hit per turn.**
Room state	<1.2 s	10 s fresh	Lighting, climate, curtains, DND per zone, plus a freshness marker
Open requests	<300 ms	—	So the agent doesn't raise a duplicate
Stay context	<1.5 s	15 min	Checkout time, room type, wifi, folio total — **tokenised, no guest name** unless consented
Preferences	<300 ms	—	Empty unless the guest opted in

> **DESIGN YOUR READS TO BE CHEAP AND BATCHED**
>
> One `room.get_state` returning everything beats five per-device reads. Every mutating tool should also return the resulting state, which removes a read-back round trip and lets the agent confirm truthfully rather than optimistically.

7. Schema and repositories

The full schema is in `db/migrations/001_init.sql`. Core tables:

```
-- catalogue
tenant            tenant_id · name · cell · kms_key_arn
property          property_id · tenant_id · timezone · default_language · config (JSONB)
room_type         (property_id, room_type_id) · manifest (JSONB) · manifest_version
room              room_id · property_id · room_number · room_type_id
                  · vendor_room_id            ← the automation system's own reference
                  · disabled_capabilities[]   ← per-room overrides
device            device_id · tenant_id · property_id · room_id
                  · certificate_serial · bound_at · binding_verified_at · revoked

-- operations
service_request   category · description · priority · status · deferred · dispatch_at · sla_due_at
fnb_menu_item     (property_id, menu_item_id) · name · keywords[] · price · available
fnb_order         items (JSONB) · total · status · folio_ref

-- guest data — SEPARATE SCHEMA, per-tenant key
guest_data.stay        stay_ref (tokenised) · checkout_at · wifi · folio
guest_data.consent     voice · preferences · personalisation
guest_data.preference  (room_id, key) · value

-- audit — append-only, WORM in production
turn_record       transcript_redacted · tool_calls (JSONB) · outcome · sealed_at
escalation        category · severity · notified[] · acknowledged_at
```

7.1 Repository interfaces

Every store sits behind an interface with an in-memory implementation, which is why the whole system runs with no database. Adding Postgres is implementing the same interface.

```ts
export interface CatalogRepository {
  getProperty(propertyId: string): Promise<PropertyConfig | undefined>;
  getRoom(roomId: string): Promise<RoomRecord | undefined>;
  getDevice(deviceId: string): Promise<DeviceRecord | undefined>;
  listRooms(propertyId: string): Promise<RoomRecord[]>;
  upsertProperty(cfg: PropertyConfig): Promise<void>;
  upsertRoom(room: RoomRecord): Promise<void>;
  upsertDevice(device: DeviceRecord): Promise<void>;
}
```

> **ONE THING TO FIX WHILE YOU ARE IN THERE**
>
> All state is currently held in in-process `Map`s — sessions, device shadow, pending confirmations, catalogue cache. That means **the service runs as exactly one instance**: two replicas would each hold different conversation history, and a payment confirmation minted on one could not be redeemed on the other.
>
> Moving these to Postgres and Redis behind the existing interfaces is what unlocks horizontal scaling. It is a prerequisite for production, not an optimisation.

8. Tenant isolation

Row-level security is enabled on every tenant-scoped table. It is a **backstop**, not the primary control — the primary control is that every query goes through a repository carrying the tenant. RLS is what saves you the day someone forgets.

```
-- app.tenant_id is set per connection by the repository layer
CREATE POLICY tenant_isolation ON service_request
  USING (tenant_id = current_setting('app.tenant_id', true));
```

> **THE TEST TO WRITE**
>
> For every store: "*if a query here forgot its tenant filter, what would happen?*" If the answer is "it returns another hotel's data", you need the backstop. Build a CI test that runs representative queries under a tenant-scoped role and asserts zero cross-tenant rows — this is the first control a hotel chain's penetration test will probe.

9. Caching and freshness

Device state is the interesting one. The automation system is the source of truth, but reading it every turn is too slow and too fragile.

```
TRUTH                SHADOW                  CACHE              AGENT
─────────            ──────────              ──────────         ──────────
head-end  ──push──►  edge local  ──stream──► cloud state ──►    room.get_state
          ◄─write──  shadow      ◄─command── service

fresh   (<10s)     → serve from shadow, no read
stale   (10–120s)  → serve from shadow, but verify-read after any write
expired (>120s)    → force a synchronous read; if that fails return UNKNOWN
                     and the agent says "I'm not sure what it's set to right now"
                     rather than confidently reporting a stale number
```

Surface the freshness marker in any read tool you build. An honest "unknown" is far better than a confident stale value — the guest can hear the difference when the room doesn't match.

PART III

Capabilities

10. The capability model

This is the abstraction that makes "a new hotel is configuration" true. Three levels:

Level	What it is	Who sees it
Capability	An abstract, vendor-neutral ability with a typed contract — `climate.setpoint` with a range, `curtains.position` 0–100	Defined once, globally, versioned
Capability manifest	Per room type: which capabilities exist, which zones, what bounds	Authored at onboarding. Generates the model's tool list
Driver mapping	Per vendor: capability operation → concrete protocol operation	**Only the adapter.** Never the model, the prompt, the tests or the fast path

10.1 The same request, three rooms

```
GUEST SAYS          AGENT SEES                  MANIFEST SAYS                ADAPTER DOES
"close the          curtains.set({              room 511 has                 inncom-adapter:
 curtains"           zone:"bedroom",             curtains.position            → object 0x2A
                     position: 0 })              driver: inncom               → SET_LEVEL 0
                                                                              → poll until confirmed
                                                                              → { position: 0 }

same words,         IDENTICAL tool call         room 88 has                  interel-adapter:
different hotel                                  curtains.open_close          → PUT /rooms/88/blinds
                                                 (binary, no position)         { state: "closed" }
                                                 driver: interel              → { position: 0 }

same words,         TOOL NOT PRESENT            room 412 has no              (nothing — the model
no motorised         in the tool surface         curtain capability           could not have
curtains             at all                                                   proposed it)
```

> **WHY ABSENCE BEATS REFUSAL**
>
> A capability the room lacks produces **no tool**. The model cannot propose it, so no runtime check is needed and no apology is generated for something that was never possible.
>
> Verified in the running build: a Deluxe room sees **18 tools**, a Suite sees **21**, and a Suite with a disabled curtain motor sees **20**. Same code, same prompts, different config.

11. Writing a manifest

```
{
  "id": "rt_suite",
  "count": 20,
  "capabilityManifest": {
    "version": "3",
    "zones": ["bedroom", "living", "bathroom"],
    "capabilities": [
      { "capability": "lighting.on_off",     "zones": ["bedroom","living","bathroom"] },
      { "capability": "lighting.brightness", "zones": ["bedroom","living"] },
      { "capability": "lighting.scene",     "zones": ["living"] },
      { "capability": "climate.setpoint",   "zones": ["bedroom","living"],
        "minC": 18, "maxC": 26 },
      { "capability": "climate.mode",       "zones": ["bedroom","living"] },
      { "capability": "curtains.position",  "zones": ["bedroom","living"] },
      { "capability": "dnd" },
      { "capability": "tv.power_volume_input" },
      { "capability": "media.playback" }
    ],
    "scenes": ["sleep", "welcome", "work", "movie"]
  }
}
```

11.1 The capability vocabulary

Capability	Value	Notes
`lighting.on_off`	boolean	Per zone
`lighting.brightness`	0–100	Implies on_off
`lighting.scene`	scene name	Names come from `scenes`
`climate.setpoint`	°C	`minC` / `maxC` enforced by policy, not schema
`climate.mode`	auto|cool|heat|fan|off	—
`curtains.position`	0–100	Continuous
`curtains.open_close`	boolean	Binary. Same tool; the adapter degrades (§15)
`dnd`	off|dnd|make_up_room	Room-level, no zone
`tv.power_volume_input`	mixed	—
`media.playback`	play|pause|next|previous	—

11.2 Per-room overrides

```
"roomOverrides": [
  { "roomRef": "r_512", "disable": ["curtains.position"],
    "reason": "motor awaiting replacement" }
]
```

> **OVERRIDES MAY ONLY REMOVE, NEVER ADD**
>
> Enforced in code. If an override could grant a capability, a config edit would widen what the agent can do without a manifest version bump — and capability changes need to be reviewable. To add something, change the room type or move the room to a different one.

12. Property configuration

One document per hotel. This is the artefact onboarding produces, and it is what makes a new property a configuration exercise.

```
{
  "propertyId": "p_blr_whitefield",  "tenantId": "t_demo_apac",
  "timezone": "Asia/Kolkata",       "languages": ["en-IN","hi-IN"],

  "automation": {
    "vendor": "inncom",
    "roomRefMapping": "vendor_room_id"   // NOT the hotel room number
  },

  "policy": {
    "climate": { "minC": 18, "maxC": 26, "maxChangesPerHour": 6 },
    "quietHours": { "start": "22:00", "end": "07:00",
                    "deferNonUrgentDispatch": true },
    "spend": { "perStayCap":  { "amount": 15000, "currency": "INR" },
               "perOrderCap": { "amount":  6000, "currency": "INR" } },
    "tierCeiling": "T3",       // kill switch: "T0" = read-only mode
    "disabledTools": []         // per-tool kill switch
  },

  "escalation": { "emergencyNumberDisplay": "112" },
  "roomTypes": [ ... ],
  "roomOverrides": [ ... ]
}
```

> **THE VENDOR ROOM REFERENCE TRAP**
>
> `vendorRoomId` (e.g. `INN-0412`) is the automation system's own identifier. Assuming it equals the hotel room number is a classic onboarding bug whose symptom is **actions landing in the wrong room** — and it can look correct in testing if the test rooms happen to line up. Verify the mapping explicitly during onboarding, room by room.

PART IV

Automation adapters

13. The contract

```ts
export interface AutomationAdapter {
  readonly vendor: string;

  connect(binding: PropertyBinding): Promise<ConnectionHandle>;
  health(): Promise<{ reachable: boolean; latencyMs: number; lastError?: string }>;

  readRoomState(room: VendorRoomRef): Promise<CapabilityState[]>;
  subscribe(room: VendorRoomRef, cb: (change: StateChange) => void): Unsubscribe;

  /** Capability in, capability out. Vendor vocabulary never escapes this file. */
  applyCapability(
    room: VendorRoomRef,
    op: CapabilityOperation,     // { capability, zone?, value }
    idempotencyKey: string,
  ): Promise<CapabilityResult>;  // { status, observed?, message? }

  /** Run at onboarding and nightly against a designated test room. */
  conformanceSuite(): ConformanceTest[];
}
```

Six methods. Everything else in the platform is built on them.

14. Writing an adapter

```ts
export class InncomAdapter implements AutomationAdapter {
  readonly vendor = 'inncom';

  private objectMap!: ObjectMap;   // per-installation, from config — NOT hardcoded
  private conn!: HeadEndConnection;

  async connect(binding: PropertyBinding) {
    // Credentials come from the secret store, scoped to ONE property.
    // Never a shared credential across properties — that turns one hotel's
    // incident into everyone's.
    const creds = await secrets.get(binding.secretRef!);
    this.objectMap = await loadObjectMap(binding.propertyId);
    this.conn = await HeadEnd.open(creds);
    return { propertyId: binding.propertyId, connectedAt: Date.now(),
             close: () => this.conn.close() };
  }

  async applyCapability(room, op, idempotencyKey) {
    // 1. Idempotency — replays must not actuate twice.
    const prior = this.seen.get(idempotencyKey);
    if (prior) return prior;

    // 2. Translate capability → vendor. THE ONLY PLACE THIS HAPPENS.
    const target = this.objectMap.resolve(room.vendorRoomId, op.capability, op.zone);
    if (!target) {
      return { status: 'rejected',
               message: `No ${op.capability} in room ${room.vendorRoomId}` };
    }

    // 3. Actuate.
    await this.conn.setLevel(target.objectId, encodeValue(op, target));

    // 4. Read back. "Applied" must mean OBSERVED applied (§17).
    const observed = await this.readCapability(room, op.capability, op.zone);
    const result: CapabilityResult = observed
      ? { status: 'applied', observed }
      : { status: 'pending' };

    this.seen.set(idempotencyKey, result);
    return result;
  }
}
```

14.1 Vendors to design around

Vendor	Typical surface	What shapes the adapter
Honeywell INNCOM	On-prem gateway, proprietary LAN protocol; sometimes a BACnet or REST bridge	LAN-only. Object/register addressing. Older installs dislike polling and have low concurrency. Firmware varies by install year — keep the object map in config, never in code.
Interel	On-prem controller, IP/REST or proprietary	More modern, still LAN-resident. Addresses rooms by *its own* reference.
BACnet/IP	Standard objects	Standardised but vendor-specific naming. **Respect write priority arrays** or you will fight the building's own control logic.
KNX	KNXnet/IP gateway	Group addresses carry no semantics without the ETS project file. Import it at onboarding to generate the mapping — manual mapping does not scale.
Lutron / Crestron	Processor with an IP protocol	Excellent lighting, but integration ports are licence-gated. **Confirm licensing during pre-sales** — it is a commercial blocker discovered late otherwise.

15. Capability degradation

Rooms differ in what their hardware can do. **The adapter absorbs that, not the model.**

```
case CapabilityId.CurtainsPosition: {
  const requested = clamp(Number(op.value), 0, 100);

  // A binary motor snaps to fully open or fully closed. The model asked for a
  // position and gets a truthful observed value back — it never learns why.
  const binary = this.objectMap.isBinaryCurtain(room.vendorRoomId, op.zone);
  const applied = binary ? (requested >= 50 ? 100 : 0) : requested;

  await this.conn.setLevel(target.objectId, applied);
  return { status: 'applied',
           observed: { capability: op.capability, zone: op.zone,
                       value: applied, available: true } };
}
```

Return the *observed* value, not the requested one. The agent confirms from what it observes, so a guest asking for 40% in a binary room hears "curtains closed" — which is true — rather than "set to 40%", which would not be.

16. Failure, retries, circuit breakers

Situation	Behaviour
Head-end slow	Hard 2.5 s budget. Beyond it the result is `pending`, and the guest is told "I've asked for that" — never "done".
Head-end unreachable	Return an error. The platform normalises it to `TEMPORARILY_UNREACHABLE` and offers the front desk.
Repeated failures	**Circuit breaker per (property, vendor)**: 5 failures in 30 s opens it for 60 s.
Retry	Automatic only for reads and reversible physical actions. Never for anything creating a commitment.
Idempotency	Honour the key. Retries happen; double actuation must not.

> **THE CIRCUIT BREAKER IS THE MOST IMPORTANT RESILIENCE CONTROL IN THE SYSTEM**
>
> One hotel's hung head-end must never consume the adapter's capacity for every other property. Without per-property bulkheading, the single most likely production incident is: one flaky BMS takes down voice control for every hotel on the platform.
>
> This is also the reason adapters are separate deployable services rather than modules.

17. Verify-read

> **NEVER REPORT SUCCESS YOU DID NOT OBSERVE**
>
> The most damaging failure mode in this product is the confident false confirmation: the guest hears "I've turned off the lights", the lights stay on, and the device is dead to them.
>
> Real systems *acknowledge writes that never take effect* — the simulator models this deliberately as "silent failure". The only defence is reading the state back.

Return	When	Guest hears
`applied`	Read-back confirms the new value	"I've set it to 23 degrees."
`pending`	Acknowledged but not yet observed	"I've asked for that — give it a moment."
`rejected`	The system refused it	"That's not something I can do here."
throw	Unreachable	"I can't reach the room systems right now."

PART V

On premises

18. Why the edge gateway exists

Four reasons, and they are not negotiable:

- INNCOM and Interel head-ends are **LAN-resident**, typically on an isolated building network with no outbound route and certainly no inbound one.
- Hotel IT **will not open inbound firewall ports** for a vendor — and the request alone delays onboarding by months.
- Round-tripping every light command to a cloud region adds 60–200 ms to an action whose entire budget is under a second.
- When the property WAN drops — routine — the guest must still be able to turn on the lights. Only something inside the building can deliver that.

```
PATH 1 — VOICE (normal turn)      device ═══► cloud
   The device's connection merely TRANSITS the hotel network. The edge gateway
   is NOT in this path. This is commonly misread from the architecture diagram.

PATH 2 — ACTUATION                cloud ──► (outbound tunnel) ──► edge ──LAN──► head-end
   The tunnel was opened OUTWARD by the edge. Nothing ever connects inward.

PATH 3 — OFFLINE ONLY             device ──LAN──► edge ──► head-end
   The only time the device talks to the edge.
```

CURRENT IMPLEMENTATION GAP

In this build the cloud creates its own adapter instance and talks to the head-end **directly** — Path 2 is not wired through the tunnel. The edge gateway exists, works and has passing tests, but is not in the actuation path.

Path 3 has the mirror gap: the appliance opens no listening socket, so a device has nothing to reach when the WAN drops. Both are Phase-2 work and both need planning now.

19. What runs on it

```
┌──────────── EDGE GATEWAY (per property; HA pair above ~250 rooms) ────────────┐
│  HARDWARE  fanless x86/ARM · 4–8 GB · 128 GB SSD · dual NIC · TPM             │
│            optional LTE failover                                              │
│                                                                               │
│  tunnel-agent     persistent OUTBOUND mTLS. No listening ports.               │
│  protocol-driver  INNCOM / Interel / BACnet / KNX / Modbus termination        │
│  state-shadow     local current state per room; publishes deltas upstream     │
│  fallback-engine  offline rules: local grammar → direct actuation             │
│  outbox           durable queue for tickets raised while the WAN was down     │
│  health-agent     metrics, remote diagnostics, watchdog                       │
└───────────────────────────────────────────────────────────────────────────────┘

 Security: per-property X.509 · secure boot · encrypted disk · outbound 443 only
           no SSH in production · property-scoped credentials — a compromise
           reaches ONE hotel
```

PART VI

Practical

20. The simulator

You can build and test a complete adapter integration **without any hotel hardware**. The simulator models the behaviours that actually bite.

```
const adapter = new SimulatorAdapter({
  latencyMs: 120,           // base write latency, ±40% jitter
  failureRate: 0.05,        // 5% of calls fail as unreachable
  silentFailureRate: 0.02,  // acknowledged but NOT applied — the nastiest real bug
  driftRate: 0.01,          // state changes because a housekeeper used a wall switch
  binaryCurtainRooms: ['INN-0088'],   // exercise capability degradation
  seed: 42,                 // deterministic — evals must be reproducible
});
```

> **BUILD AGAINST THE SIMULATOR FIRST, DELIBERATELY**
>
> Access to real hotel hardware is the biggest velocity constraint on this project, and you cannot ask a hotel to unplug its head-end to test your error handling. Every failure path in your adapter should be exercised here before you touch a real system — especially silent failure and drift, which are the two you will otherwise discover in production.

21. Conformance testing

Every adapter ships a suite that runs at onboarding and nightly against a designated test room. This is the go-live gate for a property.

```
conformanceSuite(): ConformanceTest[] {
  return [
    check('lights on',        'lighting.on_off',     true, 'bedroom'),
    check('lights dim to 40', 'lighting.brightness', 40,   'bedroom'),
    check('setpoint 23',      'climate.setpoint',    23,   'bedroom'),
    check('mode cool',        'climate.mode',      'cool',  'bedroom'),
    check('curtains closed',  'curtains.position',    0,   'bedroom'),
    check('dnd on',           'dnd',               'dnd'),
  ];
}
```

```
npm run eval -- capability     # capability matrix across room types
npm test                       # everything, including the safety gates
```

22. Onboarding a property

Target: **five working days** for a 300-room property, once an adapter for its vendor exists.

Day	Step	Detail
1	Survey	Which vendor, which firmware, which integration surface. Room types and what each actually has. Network: is there an IoT VLAN?
1–2	Room reference mapping	Hotel room number → `vendorRoomId`. **Verify, don't assume.** Import the ETS project for KNX; export the object map for INNCOM.
2	Manifests	One per room type — typically six to eight. Plus overrides for rooms with known faults.
2	Property config	Temperature band, quiet hours, spend caps, emergency number, languages, brand tone.
3	Edge appliance	Install, bind to the property, verify the outbound tunnel, confirm it reaches the head-end.
3–4	Conformance	Run the suite against a designated test room. Every capability in every room type.
4	Device enrolment	Provision hubs, bind each to its room, **verify the binding** against the automation system.
5	Content & go-live	Menu with spoken aliases, property FAQ, staff escalation targets, welfare protocol. Canary a floor before the property.

> **THE STEP THAT GOES WRONG**
>
> Room reference mapping. It is tedious, it looks trivial, and getting it wrong means actions land in the wrong room while every log says they succeeded. Budget real time for verification, and re-verify after any refurbishment — a device moved between rooms produces exactly the same silent failure.

23. Checklist

Area	Check
MCP tools	No tool takes a room/property/tenant parameter · scope honoured out of band · cross-tenant reach impossible · returns within 2.5 s · idempotent mutations · structured errors · reads are side-effect free
Data	Device resolution cached · `vendorRoomId` verified per room · manifests per room type not per room · RLS enabled and CI-tested
Adapter	Vendor vocabulary never escapes the file · object map in config not code · idempotency honoured · degradation handled in the adapter · returns *observed* values · per-property credentials
Reliability	2.5 s budget · circuit breaker per (property, vendor) · verify-read on every write · `pending` distinguished from `applied`
Testing	Conformance suite passes · silent failure exercised · drift exercised · binary degradation exercised · deterministic seed